



Task 1

Task 2

Challenge 1 description

You are given two arrays, A and B, each made of N integers. They represent a grid with N columns and 2 rows, where A is the upper row and B is the lower row.

Your task is to go from the upper-left cell (represented by A[0]) to the bottom-right cell (represented by B[N-1]) moving only right and down, so that the maximum value over which you pass is as small as possible.

Write a function:

```
int solution(int A[], int B[], int N);
```

that, given two arrays of integers, A and B, of length N, returns the maximum value on the optimal path.

Examples:

1. Given A = [3, 4, 6], B = [6, 5, 4], the function should return 5. The optimal path is 3 → 4 → 5 → 4.

3	4	6
6	5	4



2. Given A = [1, 2, 1, 1, 1, 4], B = [1, 1, 1, 3, 1, 1], the function should return 2. The optimal path is shown in the image below.

1	2	1	1	1	4
1	1	1	3	1	1

3. Given A = [-5, -1, -3], B = [-5, 5, -2], the function should return -1. The optimal path is (-5) → (-1) → (-3) → (-2).

Solution C

```
1 // you can
2 // print
3
4 int solution
5 // In
6 }
7
```

Test output



Challenge 1 description

You want to visit your friend, who lives abroad. It is time to plan the whole journey, both there and back. The trip will be long, so you would like to make it interesting. You do not want to revisit the same places or go along the same paths twice. Also, you do not have much time, so you do not want to head back from any point.

You will represent your planned path by a string containing four letters: 'N' for north, 'S' for south, 'E' for east and 'W' for west. For example, a path going north, east, east, north, west, south would be notated as "NEENWS".

You have already made a plan of the outward part of your journey. How will you plan the shortest path back home, fulfilling the criteria described above?

Write a function:

```
string solution(string &forth);
```

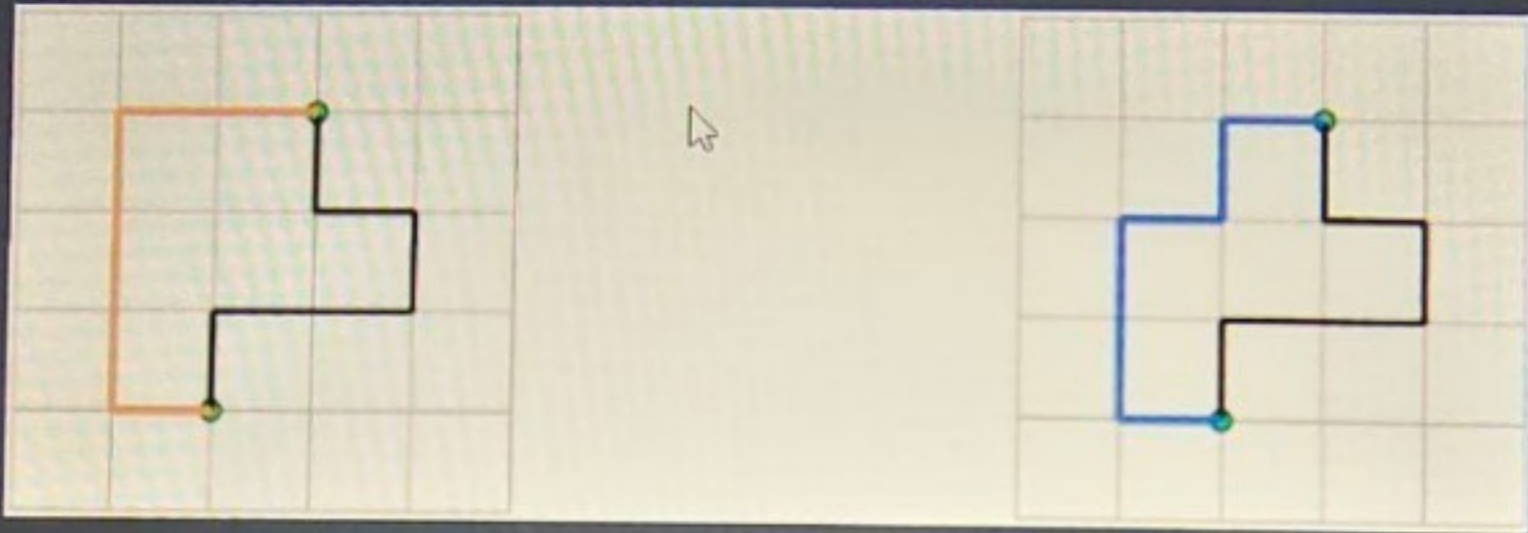
that, given a string forth of length N, which denotes the path leading to your friend, returns one of the shortest possible paths back home and fulfils the above conditions. You can assume that you are heading north at both the beginning and the end of the first part of your journey (the first and the last element in forth are equal to 'N'). Moreover, forth does not contain any occurrence of the letter 'S'.

Examples:

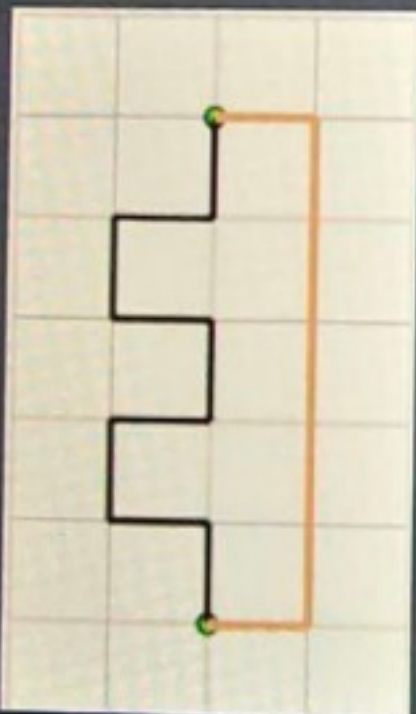
occurrence of the letter 'S'.

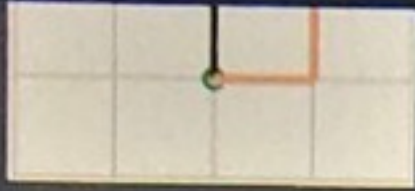
Examples:

1. Given forth = "NEENWN", your function may return "WWSSSE". It may also return "WSWSSE".

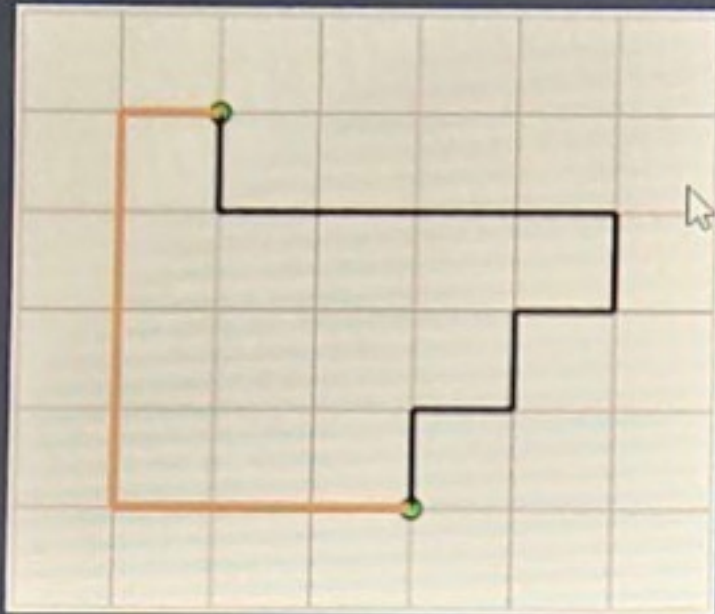


2. Given forth = "NWNENWNEN", your function may return "ESSSSSW".





3. Given forth = "NENENWWWN", your function may return "WSSSSEEE".



Write an efficient algorithm for the following assumptions:

- N is an integer within the range [2..100,000];
- string forth consists only of the following characters: 'N', 'E' and/or 'W';
- forth[0] = forth[N-1] = 'N';
- route designated by forth never goes along the same path twice.

Challenge 1 description

There is a metro rail line with k stations numbered from 0 to $k-1$. There is a direct connection between stations if their numbers differ by one. Passengers can ride the metro line in both directions.

The fee for boarding the train is 1. Every time a passenger travels between two adjacent stations the fee increases by 2. The metro system charges money at the end of each day based on the passenger's travel history for that day.

There is a total fee limit for a single day. The passenger cannot be charged more than the limit, even if their associated travel fees exceed the limit. The limit depends on the maximum station number the passenger visited during the day.

You are given arrays `start` and `dest` of length N , describing all the metro rides the passenger took during the day. During the x -th ride, the passenger boarded the train at station `start[x]` and left the train at station `dest[x]`. You are also given an array `limit` of length k . If the largest station number the passenger visited during the day is y , then the fee limit for this passenger for that day is `limit[y]`.

Write a function:

```
int solution(vector<int> &start, vector<int> &dest,
            vector<int> &limit);
```

that, given the arrays `start` and `dest`, both of length N , and the array `limit` of length k , returns the amount of money the passenger will be charged at the end of the day.

Examples:

1. For `start = [1, 0, 2, 4]`, `dest = [2, 2, 0, 5]` and `limit = [3, 17, 7, 4, 5, 17]` the function should return 16.

Solution

C++

```

8         return b;
9     }
10    int max_fun(int s, int b){
11        int m = 0;
12
13
14
15
16
17
18
19
20
21        int i;
22
23
24        int arr_cost, _st_from;
25    }
26
```

Test output

Example test: example1

Test result: OK

Example test: example2

Test result: OK

Example test: example3

Challenge 2 description

You have M square tiles of size 1×1 and N square tiles of size 2×2 . Your task is to create the largest possible square using these tiles. Tiles may not overlap, and the resulting square should be filled (it should not contain empty spaces).

Write a function:

```
int solution(int M, int N);
```

that, given two integers M and N , returns the length of the side of the largest square you can create. If no square can be created, your function should return 0.

Examples:

1. Given $M = 8$ and $N = 0$, your function should return 2. You can use four out of eight tiles to arrange them into 2×2 square. There are not enough tiles to create 3×3 square.
2. Given $M = 4$ and $N = 3$, your function should return 4. You can obtain a 4×4 square by arranging four 1×1 tiles into a 2×2 square, and surrounding it by 2×2 tiles:



3. Given $M = 0$ and $N = 16$, your function should return 8. You need to use sixteen 2×2 tiles to create the square. Note that not all the tiles are used.
4. Given $M = 13$ and $N = 3$, your function should return 5. One of the possible arrangements is shown in the following image:



Write an efficient algorithm for the following assumptions:

- M and N are integers within the range $[0..1,000,000,000]$.



Task 1

Task 2

Challenge 2 description

You are given two arrays, A and B, each made of N integers. They represent a grid with N columns and 2 rows, where A is the upper row and B is the lower row.

Your task is to go from the upper-left cell (represented by A[0]) to the bottom-right cell (represented by B[N-1]) moving only right and down, so that the maximum value over which you pass is as small as possible.

Write a function:

```
int solution(vector<int> &A, vector<int> &B);
```

that, given two arrays of integers, A and B, of length N, returns the maximum value on the optimal path.

Examples:

1. Given A = [3, 4, 6], B = [6, 5, 4], the function should return 5. The optimal path is 3 → 4 → 5 → 4.

3	4	6
6	5	4



2. Given A = [1, 2, 1, 1, 1, 4], B = [1, 1, 1, 3, 1, 1], the function should return 2.

3	4	6
6	5	4

2. Given $A = [1, 2, 1, 1, 1, 4]$, $B = [1, 1, 1, 3, 1, 1]$, the function should return 2. The optimal path is shown in the image below.

1	2	1	1	1	4
1	1	1	3	1	1

3. Given $A = [-5, -1, -3]$, $B = [-5, 5, -2]$, the function should return -1. The optimal path is $(-5) \rightarrow (-1) \rightarrow (-3) \rightarrow (-2)$.

-5	-1	-3
-5	5	-2

SOLN

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of arrays A and B is an integer within the range $[-1,000,000,000..1,000,000,000]$.

```
#include <bits/stdc++.h>
using namespace std;

int dp[3][100002];

int solve(int r, int c, vector<vector<int>> ar, int n){
    if(c==n-1 && r==1) return ar[r][c];
    if(dp[r][c] != -1) return dp[r][c];

    int right = INT_MAX, down = INT_MAX;
    if(c+1<n) right = ar[r][c] + solve(r, c+1, ar, n);
    if(r==0) down = ar[r][c] + solve(r+1, c, ar, n);

    return dp[r][c] = min(right, down);
}

int solution(int A[], int B[], int N){
    memset(dp, -1, sizeof dp);
    vector<vector<int>> ar(2);
    for(int i=0; i<N; i++){
        ar[0].push_back(A[i]);
        ar[1].push_back(B[i]);
    }

    return solve(0, 0, ar, N);
}

int main() {
    // int a[] = {1,2,1,1,1,4};
    // int b[] = {1,1,1,3,1,1};
    int a[] = {-5,-1,-3};
    int b[] = {-5,5,-2};
    int n = 3;
    cout<<solution(a,b,n)<<endl;

    // for(int i=0; i<n; i++){
    //     cout<<dp[0][i]<<" ";
    // }
    // cout<<endl;
    // for(int i=0; i<n; i++){
    //     cout<<dp[1][i]<<" ";
    // }
    return 0;
}
```


Challenge 2 description

You are given two arrays, A and B, each made of N integers. They represent a grid with N columns and 2 rows, where A is the upper row and B is the lower row.

Your task is to go from the upper-left cell (represented by A[0]) to the bottom-right cell (represented by B[N-1]) moving only right and down, so that the maximum value over which you pass is as small as possible.

Write a function:

```
int solution(vector<int> &A, vector<int> &B);
```

that, given two arrays of integers, A and B, of length N, returns the maximum value on the optimal path.

Examples:

1. Given A = [3, 4, 6], B = [6, 5, 4], the function should return 5. The optimal path is 3 → 4 → 5 → 4.

3	4	6
6	5	4

2. Given A = [1, 2, 1, 1, 1, 4], B = [1, 1, 1, 3, 1, 1], the function should return 2. The optimal path is shown in the image below.

1	2	1	1	1	4
1	1	1	3	1	1

Solution

C++

```

16 int dp[2][N];
17 for (int i = 0; i < N; i++)
18     dp[0][i] = A[i];
19     dp[1][i] = B[i];
20
21 for (int i = 1; i < N; i++)
22     for (int j = 0; j < 2; j++)
23         dp[j][i] = min(dp[j][i-1], dp[j^1][i-1]) + A[i];
24
25 return dp[1][N-1];
26
27 }
28
29
30
31
32 return
33
34

```

Test output

Example test: example1

Test result: OK

Example test: example2

Test result: OK

Example test: example3